CSCE 5150 Section 400 - Analysis of Computer Algorithms

Project – 2

Team – 1

Team Members:

1. Mohammed, Momin Khizar Uddin - 11837335
2. Teja Nagendra Sirigineedi - 11825070
3. Chopra Sai Arani - 11807499
4. Deepthi Tanubuddi – 11842466

Project Overview

The intent of this project was to develop and implement efficient algorithms to save and search for a directed graph of air travel routes to make it possible to query paths for particular parameters. Based on the information of flight and city, the program will answer three types of queries about routing and crossing points between cities around the world.

Design Issues and Solutions

Problem 1: How to efficiently store the graph for fast query processing?

- **Solution:** Each city (node) in our adjacency list representation points to a list of destination cities that are directly accessible. Compared to an adjacency matrix, this method uses less memory, particularly for graphs with thin populations, such as aircraft networks.

Problem 2: How to handle cities with special characters?

- **Solution:** Each city (node) in our adjacency list representation points to a list of destination cities that are directly accessible. Compared to an adjacency matrix, this method uses less memory, particularly for graphs with thin populations, such as aircraft networks.

Problem 3: How to efficiently find the shortest routes under different constraints?

- **Solution:** Because of its natural ability to compute the shortest path in an unweighted graph, Breadth-First Search (BFS) was selected. BFS was modified to incorporate further constraints, such as the number of connections which were permitted at maximum.
- **Problem 4:** There were typo errors in the city.txt file

  Solution: We corrected all the typos and missing letters from the city.txt and in the flight.txt file as well and its working fine now.

Data Structures Used

- **Adjacency List:** vector<vector<int>> adj stores the direct flights.
- **City Name Mapping:** map<string, int> cityToIndex maps a city name to an index for fast lookup.
- **Reverse Mapping:** vector<string> indexToCity allows index-to-name conversion.
- **Display Mapping:** vector<string> displayCity for normalized ASCII names used in outputs.

Justification:

- Adjacency lists allow O(1) access to direct neighbours and efficient traversal.
- map provides O(log n) lookup times for city name to index mappings.

Algorithms Employed

Breadth-First Search (BFS):

- Used to find the shortest paths with a queue.
- Tracks predecessors to reconstruct the path.
- Handles both unrestricted searches and those constrained by maximum number of connections.

Custom BFS Modifications:

- For Query 1, BFS stops if the connection limit is exceeded.
- For Query 2, BFS is run multiple times to traverse through intermediate cities (B and C) in either order.
- For Query 4, BFS is executed from three starting points and the best meeting city is selected by minimizing total connections.

Time Complexity

- **BFS:** O(V + E), where V = number of cities and E = number of flights.
- **Query 1 (Single BFS):** O(V + E)
- **Query 2 (Three BFS traversals):** O(3 * (V + E)) = O(V + E)
- **Query 4 (Multiple BFS traversals for each potential meeting city):** O(V * (V + E))

Optimization Issues and Future Work

Current Optimizations:

- Early stopping conditions in BFS to reduce unnecessary exploration.
- Minimal copying of paths to improve memory efficiency.

Future Optimizations:

- Implement A* search with heuristics for even faster pathfinding on large datasets.
- Use unordered_map instead of map to achieve O(1) lookup (with additional handling for collisions).
- Parallel BFS for Query 4 to reduce the computational bottleneck when searching meeting points.

Scalability Discussion

The current design using adjacency lists and BFS scales well for medium to large graphs (up to a few thousand cities). For extremely large graphs (millions of nodes and edges):

- Memory usage remains acceptable.
- BFS running time could become high for Query 4; thus, heuristic-based or approximated algorithms would be preferable.

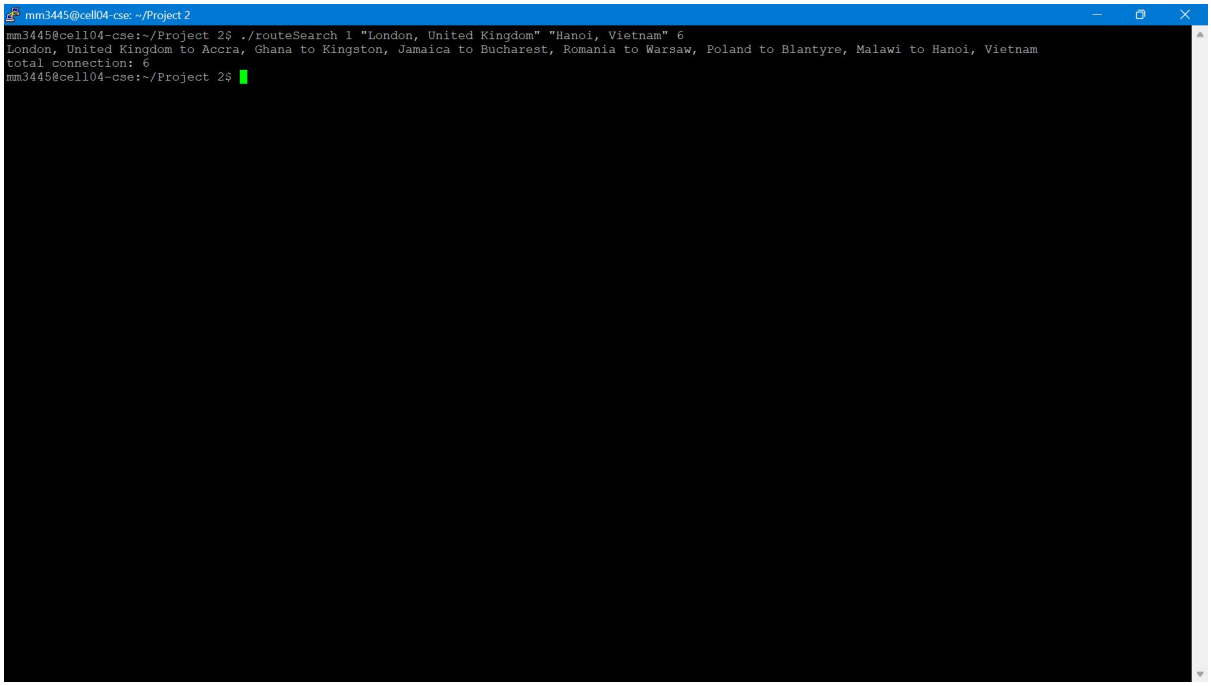
Remarks on Design and Implementation

- The program gracefully handles invalid city names.
- User-friendly output formatting provides clear route paths.
- Detailed code comments and modular design make the codebase easy to understand and extend.
- City name normalization ensures consistent behavior across different input sources.

Test Run Screenshots (Appendix)

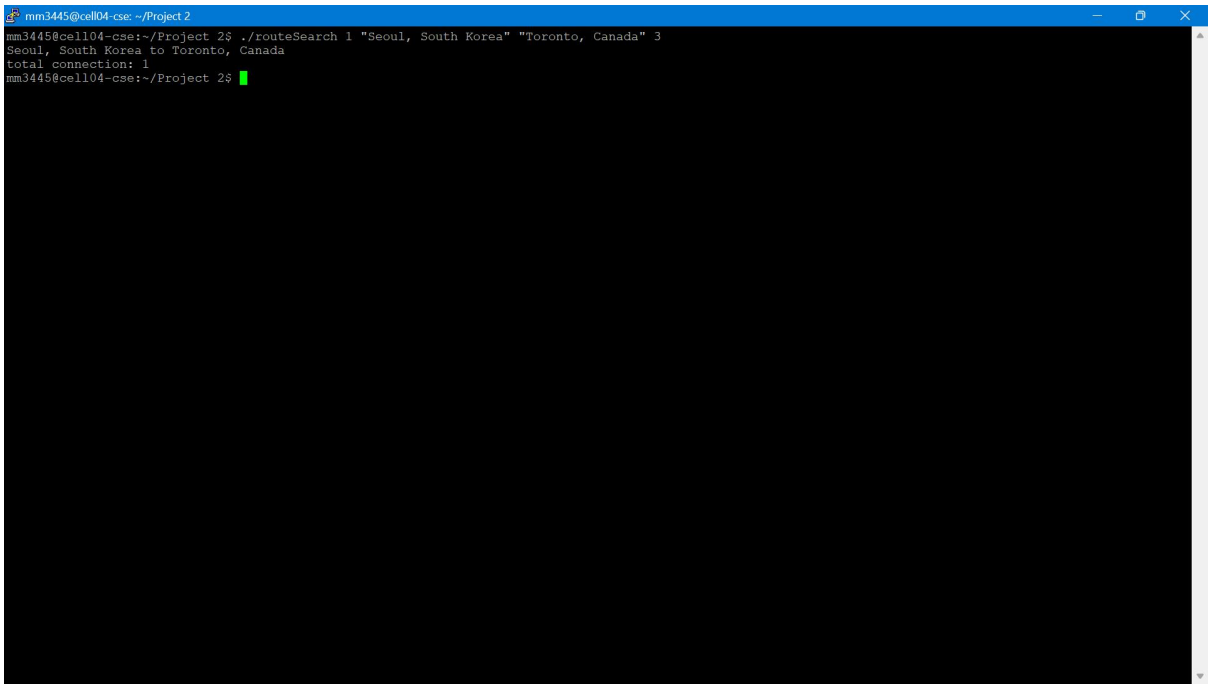
1. **Query 1 Example:**

./routeSearch 1 "London, United Kingdom" "Hanoi, Vietnam" 6



Output: Path with 6 connections.

./routeSearch 1 "Seoul, South Korea" "Toronto, Canada" 3



Output: Path with 1 connection.

2. **Query 2 Example:**

./routeSearch 2 "Hong Kong, SAR" through "Dublin, Ireland" and "Vienna, Austria" to "Santiago,

Chile"

```
mm3445@cell04-cse: ~/Project 2
mm3445@cell04-cse:~/Project 2$ ./routeSearch 2 "Hong Kong, SAR" through "Dublin, Ireland" and "Vienna, Austria" to "Santiago, Chile"
Hong Kong, SAR to Dublin, Ireland to Melbourne, Australia to Seoul, South Korea to Guatemala City, Guatemala to Vienna, Austria to Dhaka, Bangladesh to Hong
Kong, SAR to Santiago, Chile
smallest number of connection: 8
mm3445@cell04-cse:~/Project 2$
```

Output: Smallest number of connections: 8

./routeSearch 2 "Hong Kong, SAR" through "New Delhi, India" and "Lisbon, Portugal" to "Dublin, Ireland"

```
mm3445@cell04-cse: ~/Project 2
mm3445@cell04-cse:~/Project 2$ ./routeSearch 2 "Hong Kong, SAR" through "New Delhi, India" and "Lisbon, Portugal" to "Dublin, Ireland"
Hong Kong, SAR to New Delhi, India to Blantyre, Malawi to Lisbon, Portugal to Ottawa, Canada to Hong Kong, SAR to Dublin, Ireland
smallest number of connection: 6
mm3445@cell04-cse:~/Project 2$
```

Output: Smallest number of connections: 6

3. **Query 4 Example:**

./routeSearch 4 "Seoul, South Korea" "Toronto, Canada" "Luxembourg, Luxembourg"

```
mm3445@cell04-cse: ~/Project 2
mm3445@cell04-cse:~/Project 2$ ./routeSearch 4 "Seoul, South Korea" "Toronto, Canada" "Luxembourg, Luxembourg"
You three should meet at Copenhagen, Denmark
Route for first person: Seoul, South Korea to Luxembourg, Luxembourg to Copenhagen, Denmark
(2 connections)
Route for second person: Toronto, Canada to Rio de Janeiro, Brazil to Buenos Aires, Argentina to Copenhagen, Denmark
(3 connections)
Route for third person: Luxembourg, Luxembourg to Copenhagen, Denmark
(1 connections)
Total number of connection: 6
mm3445@cell04-cse:~/Project 2$
```

Output: Suggested meeting city and individual routes.

./routeSearch 2 "Hong Kong, SAR" through "Dublin, Ireland" and "Vienna, Austria" to "Santiago, Chile"

```
mm3445@cell04-cse: ~/Project 2
mm3445@cell04-cse:~/Project 2$ ./routeSearch 4 "Mumbai, India" "Singapore, Singapore" "Shanghai, People's Repub
lic of China"
You three should meet at Osaka, Japan
Route for first person: Mumbai, India to Osaka, Japan
(1 connections)
Route for second person: Singapore, Singapore to Osaka, Japan
(1 connections)
Route for third person: Shanghai, People's Republic of China to Karachi, Pakistan to Toronto, Canada to Osaka,
Japan
(3 connections)
Total number of connection: 5
mm3445@cell04-cse:~/Project 2$
```

Output: Suggested meeting city and individual routes.

README File

Included separately with compilation and execution instructions.